

AutoCarePro

REST APIs & Spring Boot Annotations

Vehicle Service Management System

TECHNOLOGY	ROLE IN YOUR PROJECT
React + Vite	Frontend UI and REST API calls
Spring Boot 4.1.1	REST API backend
Java 17	Backend implementation
Spring Data JPA / Hibernate	Repository and persistence layer
MySQL	Database, SQL, procedures, functions and triggers

API base URL: `http://localhost:8080/api`

This guide is based on the AutoCarePro frontend/API code and backend examples you shared in this conversation.

1. REST API Execution Flow

React does not directly execute your SQL. The frontend sends an HTTP request to Spring Boot. The backend processes it through the controller, service and repository layers and finally interacts with MySQL.

```
React Frontend
↓
fetch("http://localhost:8080/api/...")
↓
REST Controller
↓
Service
↓
Repository
↓
MySQL / JPA / SQL / Function / Procedure
↓
JSON Response
↓
React UI
```

Mentor answer: I exposed my database-backed operations through REST APIs. React calls the Spring Boot controllers, controllers delegate to services, services use repositories, and the result is returned as JSON.

2. CRUD APIs Used

METHOD	ENDPOINT	PURPOSE
GET	/api/customers	Fetch customers
POST	/api/customers	Create customer
PUT	/api/customers/{id}	Update customer
DELETE	/api/customers/{id}	Delete customer
GET	/api/vehicles	Fetch vehicles
POST	/api/vehicles	Create vehicle
PUT	/api/vehicles/{id}	Update vehicle
DELETE	/api/vehicles/{id}	Delete vehicle
GET	/api/mechanics	Fetch mechanics
POST	/api/mechanics	Create mechanic
PUT	/api/mechanics/{id}	Update mechanic
DELETE	/api/mechanics/{id}	Delete mechanic
GET	/api/service-jobs	Fetch service jobs
POST	/api/service-jobs	Create service job
PUT	/api/service-jobs/{id}	Update service job
DELETE	/api/service-jobs/{id}	Delete service job
GET	/api/service-types	Load service types
GET	/api/payments/job/{jobId}	Fetch payment records for a job

3. Database Feature APIs

FEATURE	REST ENDPOINT	FRONTEND FEATURE
MySQL Function	GET /api/reports/service-cost/{jobId}	Service Cost Calculator
Stored Procedure	GET /api/service-jobs/history/{vehicleId}	Vehicle Service History
JOIN	GET /api/reports/service-jobs	Service Job Overview
Subquery	GET /api/reports/mechanic-performance	Mechanic Performance

4. Spring Boot Annotations

ANNOTATION	ROLE	USE
@RestController	Controller	Exposes a Java class as a REST controller.
@RequestMapping	Controller	Defines a common URL prefix such as /api/reports.
@GetMapping	Controller	Maps an HTTP GET request to a method.
@PostMapping	Controller	Maps an HTTP POST request for create operations.
@PutMapping	Controller	Maps an HTTP PUT request for update operations.
@DeleteMapping	Controller	Maps an HTTP DELETE request.
@PathVariable	Controller	Reads a value from the URL, such as jobId.
@RequestBody	Controller	Converts incoming JSON into a Java object.
@CrossOrigin	Controller	Allows the React frontend to call the backend from another origin.
@Service	Service	Marks the service/business-layer class as a Spring bean.
@Repository	Repository	Marks the data-access/repository component.

5. Your Service Cost REST Controller

```
@RestController
@RequestMapping("/api/reports")
@CrossOrigin(origins = "")
public class ServiceCostController {

    @GetMapping("/service-cost/{jobId}")
    public ResponseEntity<?> getServiceCost(
        @PathVariable Long jobId) {

        BigDecimal totalCost =
            serviceCostService.calculateServiceCost(jobId);

        return ResponseEntity.ok(
            Map.of(
                "jobId", jobId,
                "calculatedServiceCost", totalCost
            )
        );
    }
}
```

Explanation: @RestController exposes the class. @RequestMapping sets the base path. @GetMapping defines the endpoint. @PathVariable receives jobId from the URL. The controller calls ServiceCostService and returns the result as a JSON response.

6. Frontend → Function API Flow

```
User enters Job ID = 7
↓
React fetch()
↓
GET /api/reports/service-cost/7
↓
ServiceCostController
↓
ServiceCostService
↓
ServiceCostRepository
↓
MySQL function: calculate_service_cost(7)
↓
JSON result
↓
React displays calculated cost
```

7. Frontend → Stored Procedure Flow

```
Vehicle ID entered
↓
GET /api/service-jobs/history/{vehicleId}
↓
ServiceJobProcedureController
↓
ServiceJobProcedureService
↓
ServiceJobProcedureRepository
↓
MySQL stored procedure
↓
Vehicle service history
↓
React table
```

8. Frontend → JOIN and Subquery Flow

```
JOIN:
React → GET /api/reports/service-jobs
→ Controller
→ Service
→ ServiceJobReportRepository
→ JOIN SQL
→ JSON report
→ React table

SUBQUERY:
React → GET /api/reports/mechanic-performance
→ Controller
→ Service
→ MechanicPerformanceRepository
→ Subquery
→ JSON report
→ React table
```

9. Trigger Execution

A trigger is different from a function or stored procedure. The application does not normally call a trigger directly. MySQL automatically executes the trigger when its configured INSERT, UPDATE or DELETE event occurs.

```
React
↓
REST API
↓
Spring Boot
↓
Repository
↓
INSERT / UPDATE / DELETE
↓
MySQL
↓
TRIGGER AUTOMATICALLY FIRES
↓
Trigger action
```

Mentor answer: I do not explicitly call the trigger from React. The CRUD operation reaches MySQL, and MySQL automatically fires the trigger when its condition/event is satisfied.

10. Where to Show Each Feature

FEATURE	ECLIPSE / DATABASE	FRONTEND
JOIN	ServiceJobReportRepository.java + JOIN SQL	Service Job Overview
Subquery	MechanicPerformanceRepository.java + subquery	Mechanic Performance
Function	ServiceCostRepository.java + MySQL function	Service Cost Calculator
Procedure	ServiceJobProcedureRepository.java + procedure	Vehicle Service History
Trigger	08_triggers.sql / MySQL SHOW TRIGGERS	Fires automatically during its event

11. Quick Mentor Q&A;

Where is the REST API?

In the Spring Boot controller package.

How does React call it?

Using `fetch()` with the backend URL `http://localhost:8080/api`.

Where is SQL/database logic?

In the repository/data-access layer or MySQL database objects, depending on the feature.

How is the function exposed?

The controller receives the HTTP request → service → repository → MySQL function → JSON response.

How is the procedure exposed?

The REST endpoint delegates through controller → service → repository, where the stored procedure is invoked.

How does the trigger execute?

MySQL automatically fires it after/before the database event defined in the trigger.

What does `@PathVariable` do?

It extracts a value such as `jobId` from the URL path.

What does `@RequestBody` do?

It maps incoming JSON data to the Java object used by the controller.

Final line to memorize: React → REST Controller → Service → Repository → MySQL → JSON → React UI.

Project environment noted from your shared backend logs: Spring Boot 4.1.1, Java 17.0.20.1, embedded Tomcat on port 8080, with Spring Data JPA repositories.